

1 **Automated Behavioral Regression Testing of Traffic-Signal Controllers**  
2 **Using Field-Derived Event Replay**

3 [AUTHOR NAME], [COAUTHOR NAME]  
4 [Job title], [Institution]  
5 [email address]  
6 ORCID: [optional]

7 Recommended category: traffic signal systems, operations, and control software

8 **Author review required.** Replace bracketed author, affiliation, contact, and acknowledgment text before submission.  
9 Total manuscript pages: 18

## Structured Abstract

2 **Objectives.** This study evaluates whether field-recorded controller inputs can be replayed through controller emulators  
3 to detect behavioral changes caused by a software release or an operational timing-parameter change. The transporta-  
4 tion need is a repeatable regression screen that exercises realistic combinations of detection, pedestrian, and preemption  
5 activity.

6 **Methods.** Twenty-five production configurations were replayed from approximately 23-hour field event logs. The  
7 input stream used National Transportation Communications for Intelligent Transportation Systems Protocol (NTCIP)  
8 detector, pedestrian, and preemption objects. Outputs from a MAXTIME emulator were collected as high-resolution  
9 event traces, latency-compensated, grouped into signal-state intervals, and compared with sequence and timing scores.  
10 Two comparisons were evaluated: versions 2.15.1 versus 2.18.1, and version 2.18.1 versus the same software with  
11 trailing-overlap timing parameters changed.

12 **Findings.** The software-release comparison passed 19 of 25 configurations; six were candidate behavioral differences.  
13 In the known intervention, 11 of 14 changed configurations were flagged and all 11 unchanged configurations passed.  
14 Three changed configurations passed, showing that a replay detects an intervention only when the recorded operation  
15 exercises its affected behavior.

16 **Novelty.** Prior work has automated scripted controller tests and hardware-, software-, and emulator-in-the-loop experi-  
17 ments. This paper evaluates their combination with long-duration field-derived replay, automated high-resolution trace  
18 alignment, and repeated testing across production configurations for both software- and parameter-induced changes.

19 **Practical Applications.** Agencies can use the workflow as a release and timing-change screening step before field  
20 deployment. NTCIP input replay is portable in principle; an implementation still needs an output-log adapter and event-  
21 code mapping for each controller family.

# 1 Introduction

2 Traffic-signal controller changes are often validated with short scripted checks, bench tests, or a small number of field  
3 observations. Those checks are valuable but may not exercise the combinations of detector calls, pedestrian service,  
4 preemption, coordination, and overlap behavior that occur during daily operation. A software release can therefore pass  
5 a functional checklist while changing a less common sequence. The same risk applies when the controller software is  
6 unchanged but timing parameters are edited.

7 This paper studies a complementary regression screen: replay a recorded operational event stream through a controller  
8 or emulator, collect the resulting high-resolution event trace, and compare it with a reference trace. The question is  
9 deliberately behavioral rather than vendor-specific:

10 > Can field-recorded controller inputs be replayed through controller emulators to automatically detect behavioral  
11 changes caused by either a software update or an operational timing-parameter change?

12 The implementation is an open-source Python package. Its input side uses NTCIP 1202-style detector, pedestrian, and  
13 preemption objects, so the replay concept can be adapted to compliant controllers. The evaluation used MAXTIME  
14 emulators and a MAXTIME HTTP event-log collector; those are adapters at the experiment boundary, not a claim  
15 that all NTCIP controllers work without integration. The paper contributes an auditable method and two controlled  
16 evaluations rather than a controller certification procedure.

## 17 Related Work

18 Traffic-controller automated testing has a substantial prior history. Li et al. developed a controller automated-testing  
19 tool and script language for exercising controller functions (2008). The Idaho project similarly used an automated  
20 testing tool with XML scripts, a controller interface device, and NTCIP support for selected controller classes (Ahmed  
21 et al. 2010). Tung’s Florida projects extended automated NTCIP-based testing across multiple manufacturers  
22 and emphasized the benefit of a standard interface (Tung 2012, 2015). These studies establish the value and portability  
23 of automated functional testing, but their inputs are designed scripts rather than long operational traces.

24 Hardware-in-the-loop and software-in-the-loop environments provide another foundation. Stevanovic, Klanac, and  
25 Radivojevic evaluated six controllers, event codes, and high-resolution logging in a HILS setup (2017). Wang et  
26 al. described a virtual controller interface for HILS (2019). Those studies show how controller behavior and high-  
27 resolution event data can be evaluated in controlled environments. They do not evaluate repeated release and parameter  
28 regression across numerous production configurations using field-derived inputs.

29 Dynamic time warping (DTW) provides a general foundation for aligning sequences that are similar but not  
30 synchronized sample-for-sample (Sakoe and Chiba 1978). In this project, alignment is applied after converting event  
31 records to qualified signal-state intervals and grouping adjacent event records. The cautious contribution is therefore a  
32 combination claim: the reviewed literature did not identify an evaluation combining long-duration field-derived replay,  
33 automated high-resolution output-trace alignment, and repeated production-configuration testing for both software-  
34 and parameter-induced changes. NTCIP, emulation, high-resolution logging, automated testing, and DTW are not  
35 claimed as individually novel.

## 36 Field-Event Replay and Comparison Method

### 37 Inputs and replay

38 The source data comprise 25 Parquet event logs in the repository’s validation log directory. Each file covers  
39 about 23 hours and contains a device identifier, timestamp, event identifier, and parameter. The complete source set  
40 contains 7,296,218 rows and approximately 575 source-hours. Replay selects detector, pedestrian, preemption, and  
41 related events required by the controller test configuration. The replay runner sends those events through the controller  
42 interface and records the resulting output events.

43 The package separates portable input semantics from controller-specific collection. NTCIP objects define the input  
44 vocabulary used by the replay. The evaluated MAXTIME adapter supplies output events through an HTTP event-  
45 log interface. A different controller family would require an output-collection adapter and a mapping from its event

1 codes to the common comparison vocabulary. This boundary keeps the research question manufacturer-neutral without  
 2 implying zero integration effort.

### 3 Latency and output traces

4 Controller output timestamps are not assumed to have the same delay as source-log timestamps. The 2.18.1 runs used  
 5 adaptive latency compensation: recent matched event pairs estimate an offset, and the runner updates the offset only  
 6 after a minimum sample count. The stored 2.18.1 database contains 707,458 latency samples and 6,577 offset updates.  
 7 The same compensation workflow is present in the parameter-intervention run. A fixed offset remains available for  
 8 environments that cannot estimate latency online.

### 9 Alignment and scoring

10 Events are restricted to the configured comparison identifiers and transformed into signal-state intervals. Adjacent  
 11 compatible records are grouped within a 0.25-second tolerance. Invalid, missing, or unreliable intervals are marked in  
 12 both timelines; a comparison window overlapping invalid data is excluded rather than reported as a difference. Edge-  
 13 truncated intervals and diagnostic issues without five minutes of surrounding valid context are also excluded. These  
 14 rules were added to the package with focused tests (commit 5eb309b).

15 For each qualified rolling window, the comparison aligns the reference and candidate event sequences, then reports  
 16 sequence match and timing match. Timing match is the fraction of matched events within 0.5 seconds. A configuration  
 17 passes when sequence match is at least 95 percent and timing match is at least 90 percent, with phase-call reliability  
 18 of at least 85 percent. A failed configuration is a screening candidate, not a confirmed defect: it requires review of the  
 19 aligned trace, controller logs, and configuration history.

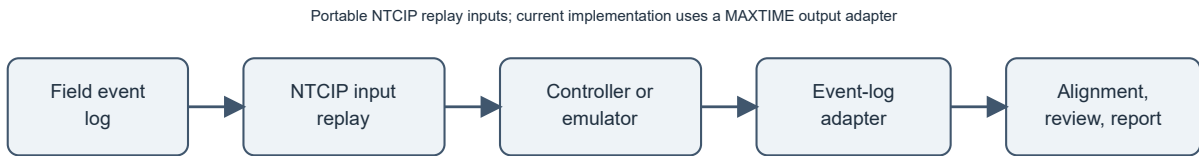


Figure 1: Workflow evaluated in this paper. Input replay is portable at the NTCIP object boundary; the current output collector is a MAXTIME adapter.

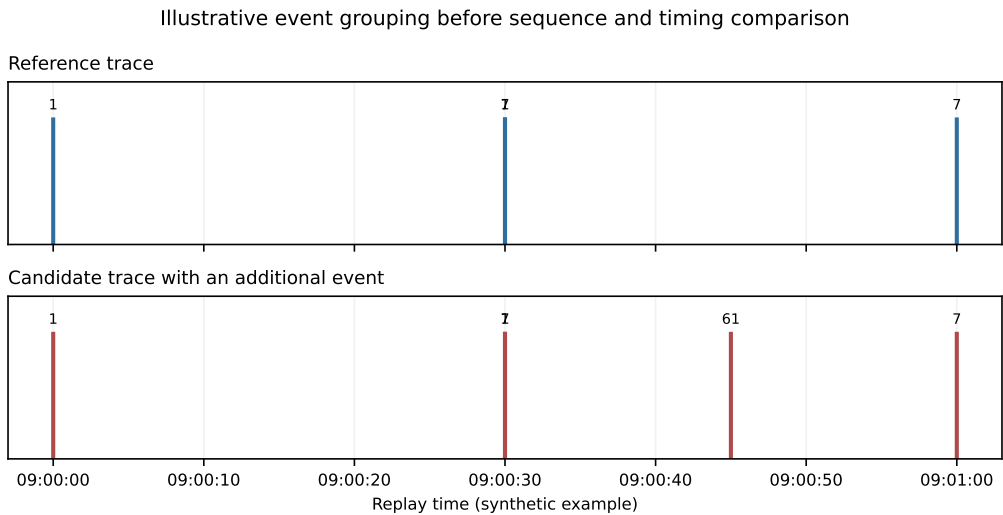


Figure 2: Illustrative alignment from the committed offline example. The synthetic candidate contains an additional included event; this figure explains grouping and comparison but is not a field result.

# 1 Experimental Design

2 The experiment uses three stored result databases. The first contains the reference 2.15.1 run, the second the 2.18.1  
3 candidate run, and the third the 2.18.1 run after trailing-overlap timing parameters were changed. All three databases  
4 contain 25 configuration identifiers and completed simulation runs. The comparison/reporting reproduction reads these  
5 databases and the archived reports; it does not start a controller, so the paper’s reproducible claim is limited  
6 to the comparison stage.

| Item                      | 2.15.1             | 2.18.1             | 2.18.1 modified    |
|---------------------------|--------------------|--------------------|--------------------|
| Configurations            | 25                 | 25                 | 25                 |
| Output events             | 7,574,994          | 8,019,167          | 7,993,042          |
| Input events              | 627,240            | 627,240            | 627,240            |
| Controller/emulator hours | about 585          | about 580          | about 577          |
| Replay source             | same 25 field logs | same 25 field logs | same 25 field logs |

| Comparison parameter    | Value        |
|-------------------------|--------------|
| Grouping tolerance      | 0.25 s       |
| Rolling window / step   | 300 s / 60 s |
| Sequence pass threshold | 95%          |
| Timing pass threshold   | 90%          |
| Timing match tolerance  | 0.50 s       |
| Phase-call reliability  | 85%          |

7 The second experiment is a known intervention. Transaction histories identify 14 configurations with trailing-overlap  
8 or associated red-revert edits and 11 configurations without those edits. The manifest records the grouping and the  
9 fields observed in transaction histories. Detailed individual edits remain marked for author verification because the  
10 exploratory conflict script assumes an overlap-parent relationship that has not been validated as a general interpretation.

## 11 Software-Release Results

12 The 2.15.1-to-2.18.1 report passed 19 of 25 configurations. The average sequence match was 96.2 percent and the  
13 average timing match was 93.8 percent. Six configurations failed at least one threshold: 12036, 2B045, 2B054, 2B085,  
14 2B094, and 2B339. These are candidate behavioral differences, not confirmed software bugs. Their causes may include  
15 changed behavior, replay exposure, data validity, or remaining integration effects.

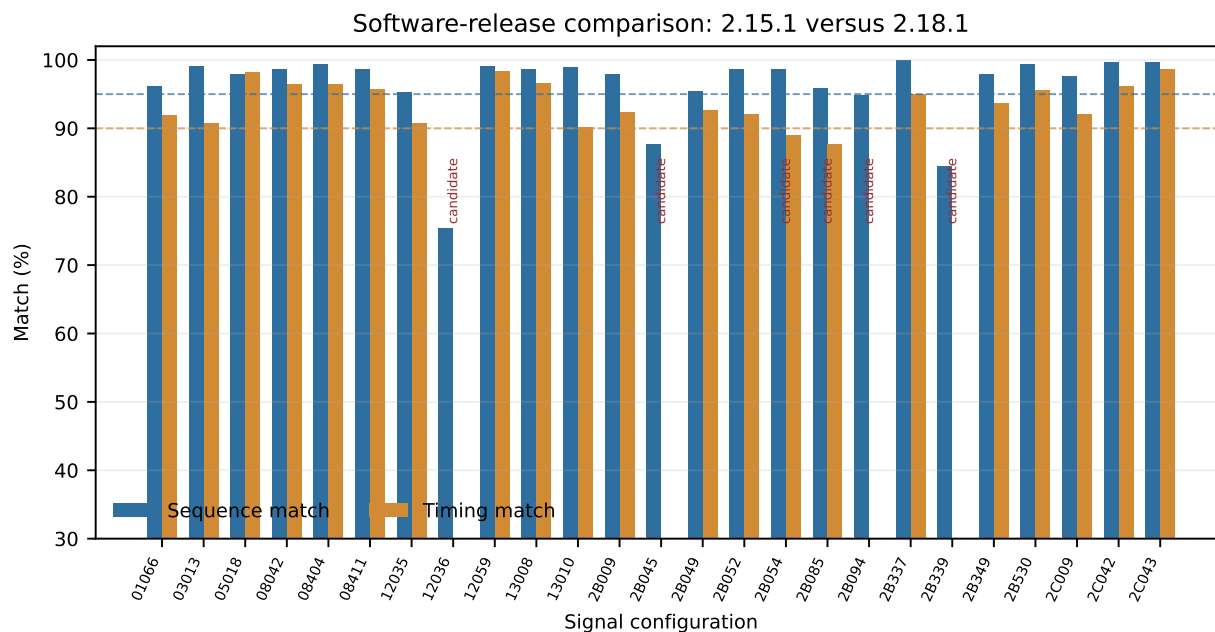


Figure 3: Sequence and timing scores for all 25 configurations in the 2.15.1-to-2.18.1 comparison. Dashed lines show the 95-percent sequence and 90-percent timing thresholds; candidate configurations require review.

| Configuration | Sequence % | Timing % | Status    |
|---------------|------------|----------|-----------|
| 01066         | 96.1       | 91.9     | Pass      |
| 03013         | 99.0       | 90.8     | Pass      |
| 05018         | 97.9       | 98.2     | Pass      |
| 08042         | 98.6       | 96.5     | Pass      |
| 08404         | 99.4       | 96.5     | Pass      |
| 08411         | 98.6       | 95.7     | Pass      |
| 12035         | 95.3       | 90.7     | Pass      |
| 12036         | 75.4       | —        | Candidate |
| 12059         | 99.1       | 98.4     | Pass      |
| 13008         | 98.6       | 96.6     | Pass      |
| 13010         | 98.9       | 90.1     | Pass      |
| 2B009         | 97.9       | 92.4     | Pass      |
| 2B045         | 87.7       | —        | Candidate |
| 2B049         | 95.4       | 92.6     | Pass      |
| 2B052         | 98.6       | 92.0     | Pass      |
| 2B054         | 98.7       | 89.0     | Candidate |
| 2B085         | 95.9       | 87.6     | Candidate |
| 2B094         | 94.8       | —        | Candidate |
| 2B337         | 99.9       | 95.0     | Pass      |
| 2B339         | 84.5       | —        | Candidate |
| 2B349         | 97.9       | 93.7     | Pass      |
| 2B530         | 99.4       | 95.6     | Pass      |
| 2C009         | 97.6       | 92.1     | Pass      |
| 2C042         | 99.7       | 96.1     | Pass      |
| 2C043         | 99.7       | 98.6     | Pass      |

# 1 Parameter-Intervention Results

2 The same-software comparison is the method's stronger validation. Of the 14 configurations with trailing-  
3 overlap or associated red-revert changes, 11 were automatically flagged and three passed. All 11 configurations without  
4 those edits passed. Thus the observed flag rate was 78.6 percent in the changed group and zero percent in the unchanged  
5 group. The three changed-and-passed cases demonstrate an important interpretation boundary: a recorded replay can  
6 detect a change only when the exercised behavior is sufficiently exposed in the trace.

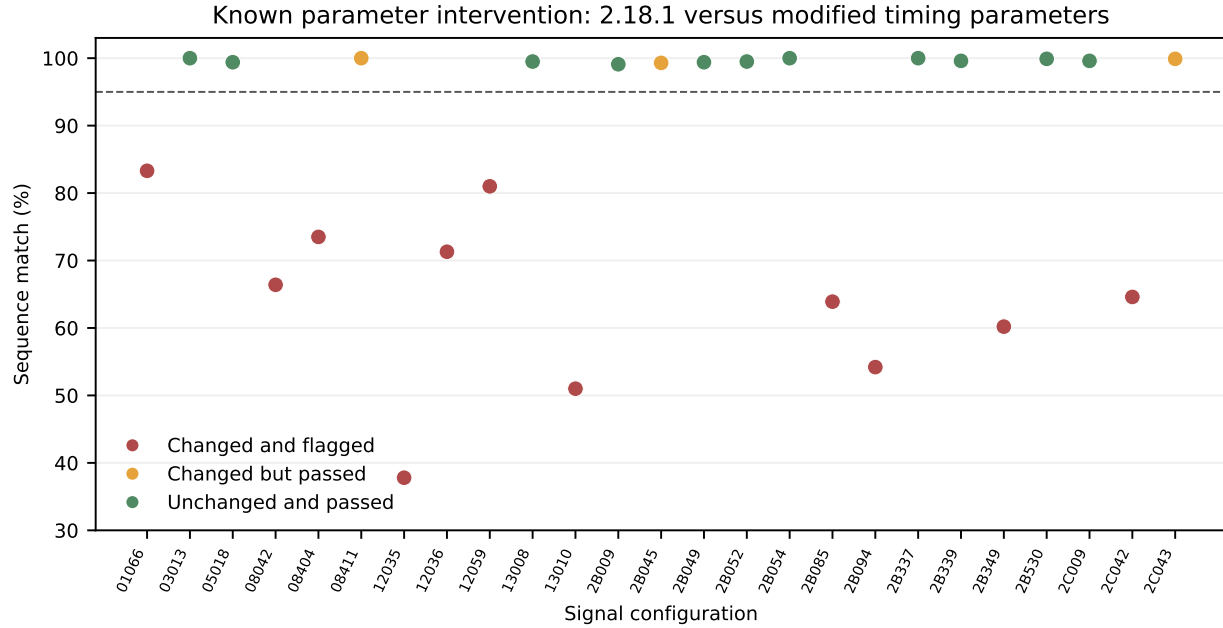


Figure 4: Known-intervention results. Red points are changed configurations flagged by the comparison; amber points are changed configurations that passed; green points are unchanged configurations that passed.

| Group                | Configurations | Flagged | Passed |
|----------------------|----------------|---------|--------|
| Changed parameters   | 14             | 11      | 3      |
| Unchanged parameters | 11             | 0       | 11     |

7 The intervention result supports a screening interpretation. It shows that the method responds to a deliberate operational  
8 change while software is held constant, and that it does not flag the unchanged group in this dataset. It does not estimate  
9 sensitivity for arbitrary timing changes, because the 14 changed configurations were selected from one transaction  
10 history and the three passed cases may not have exercised the affected overlap behavior.

## 11 Discussion and Practical Use

12 The workflow is useful when a controller agency has a reference trace, a proposed software or configuration change,  
13 and access to a controller or emulator. A release engineer can first run the replay in batch, use the pass/fail summary  
14 to triage configurations, and then inspect aligned windows only for candidates. This is more informative than a raw  
15 event-count difference because the score is tied to qualified windows, timing tolerance, and validity rules.

16 The method is manufacturer-neutral in its research framing but not adapter-free. NTCIP replay objects provide a  
17 portable input boundary. A new controller family still needs an output collector, event-code mapping, timestamp/  
18 latency behavior, and validation of its event semantics. The MAXTIME implementation demonstrates feasibility, not  
19 universal plug-and-play compatibility.

20 The open-source package is part of the contribution. Commit 5eb309b completes adaptive validation settings propa-  
21 gation, batch failure handling, and timeline reliability rules. Commit 1f25f36 documents the workflow, ignores the  
22 live settings file in favor of a tracked example, adds citation metadata, and provides a synthetic offline comparison

1 test. The package is not presented as a certified testing product; maintenance and controller adapters remain project  
2 responsibilities.

### 3 **Limitations**

4 First, the stored replay campaigns were executed with MAXTIME emulators and a MAXTIME event-log interface.  
5 Repeating the study on another controller family is a necessary external validation. Second, the field logs and config-  
6 uration databases represent one agency context and one set of operational conditions. Third, the comparison script in  
7 this paper reproduces the archived comparison/reporting stage and cross-checks database counts; it does not rerun the  
8 controller campaigns. Fourth, the detailed intervention edits need author confirmation, and the exploratory conflict  
9 CSVs are excluded. Fifth, sensitivity reruns for grouping tolerance, phase-call reliability, and timing tolerance were  
10 not available in the archived reports and are explicitly marked `not_run` in the generated sensitivity table. Finally, the  
11 six software-release candidates have not been manually classified as defects, intended changes, or data artifacts.

### 12 **Conclusions**

13 This evaluation supports a focused answer to the research question. Field-derived event replay can automatically  
14 screen controller behavior for both a software-release change and a same-software timing-parameter intervention.  
15 Across 25 configurations, the release comparison passed 19 and identified six candidates for review. In the deliberate  
16 intervention, 11 of 14 changed configurations were flagged while all 11 unchanged configurations passed. The result  
17 is a practical, reproducible comparison method and a credible basis for agency regression screening, not a claim of  
18 universal controller compatibility or automatic defect diagnosis.

19 The next high-value validation is to rerun the comparison on a second controller family through an adapter and  
20 to complete manual classification of the six release candidates and three changed-and-passed intervention cases.  
21 Publishing the comparison-only event bundle would allow independent reproduction of the alignment and reporting  
22 stage without distributing controller firmware or proprietary configuration databases.

### 23 **Acknowledgments**

24 [AUTHOR: identify agency, project, and technical support acknowledgments; confirm any required institutional  
25 approval.]

### 26 **Code and Data Availability**

27 The open-source package state used for this draft is on the repository main branch at commits 5eb309b and 1f25f36. The  
28 replay campaign still requires controller/emulator infrastructure. The repository contains an exporter for a candidate  
29 comparison-only bundle under `paper/reproducibility/`; it has not been generated or published. The author should  
30 review and approve the bundle before attaching it to GitHub, Zenodo, or another archive.

### 31 **Generative-AI Disclosure**

32 OpenAI Codex was used to inspect repository files, draft analysis scripts, generate paper figures, and draft and format  
33 this manuscript. The authors reviewed the source materials and remain responsible for the accuracy, interpretation,  
34 citations, and final submission. The author must revise this statement if the actual submission uses additional gener-  
35 ative-AI tools or materially different purposes.

### 36 **Bibliography**